# Microchip Character Recognition Using Image Processing and Machine Learning

ME5411 Robot Vision and AI — Group 8


| Name | Student number |
| --- | --- |
| Xuanwei Liu | XXX |
| Jianan Yan | XXX |
| Jiayi Guo | XXX |
| Hongchao Wang | XXX |


National University of Singapore 19 October 2026 Working report: student numbers and group-authored analysis pending.

**Abstract**

This report presents an image-processing and machine-learning pipeline for recognizing characters printed on a microchip. The report describes the preprocessing, segmentation, feature extraction, classification, and evaluation procedures developed by Group 8.

**Keywords** robot vision, image processing, character recognition, convolutional neural network, classification

# 1 Introduction

Character recognition in low-contrast industrial imagery requires careful preprocessing, segmentation, and classification. Classical image-processing methods provide interpretable tools for enhancing and isolating characters [1], while convolutional neural networks can learn features directly from image data [2].

The report follows the seven course tasks. Modules `task_01` through `task_06` implement image processing; `task_07` contains the CNN and non-CNN classifiers.

# 2 Problem Formulation

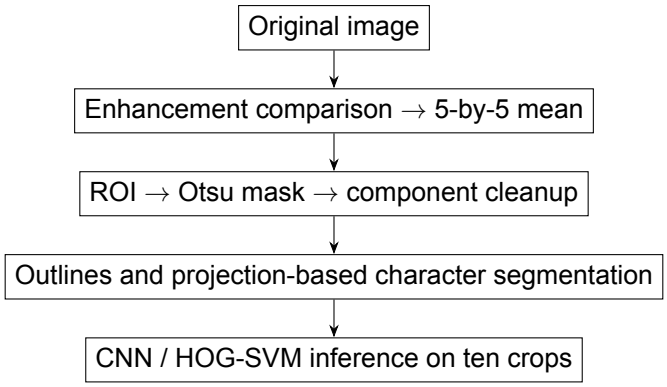
The input is a 990-by-367 grayscale microchip image. The target line is HD44780A00. Seven character classes (0, 4, 7, 8, A, D, H) are learned from the separate course classification dataset. A stratified 75/25 training/test split is fixed with seed 42. Configuration selection uses only a validation subset of the training data. Chip crops are used for inference only, never training. The official input filename differs between the assignment PDF and the local dataset; the group must confirm provenance.

# 3 Methodology

## 3.1 Tasks 1–6: Integrated Image Processing

Task 1 uses grayscale conversion, 1st–99th percentile contrast stretching, CDF-based histogram equalization and mean-centered contrast scaling. Task 2 compares normalized averaging filters of sizes 3, 5 and 7; the 5-by-5 mean output is passed downstream. Task 3 crops $(x, y, w, h) = (45, 180, 930, 180)$. Task 4 compares global Otsu and adaptive Gaussian thresholding (block size 31, constant $C = -5$). The Otsu mask is selected for subsequent processing.

Task 5 removes connected components with area below 200 pixels. Sobel gradient magnitude on the smoothed ROI is normalized and thresholded with Otsu, then masked to a dilated foreground neighborhood. An additional binary outline is obtained by subtracting a 3-by-3 erosion from the cleaned mask. Task 6 splits wide connected components at vertical-projection valleys, using median component width to estimate the number of characters. Ten boxes are expected for this specific target line. The known target text supplies manual reference labels only; it is not a recognition result.




## 3.2 Classifier

### 3.2.1 Subtask 7-1: CNN Classifier

Subtask 7-1 employs a convolutional neural network (CNN) to classify the segmented character images. The model distinguishes seven character classes: 0, 4, 7, 8, A, D, and H.

Before being passed to the network, each character image is converted to a single-channel grayscale image and resized to $32 \times 32$ pixels using nearest-neighbour interpolation. The pixel intensities are then scaled from $[0, 255]$ to $[0, 1]$. Let $I_r(u, v)$ denote the resized grayscale image. The normalized input is defined as

$$I_n(u, v) = \frac{I_r(u, v)}{255}. \tag{1}$$

The feature-extraction stage contains three convolutional layers with $3 \times 3$ kernels. The first convolutional layer converts the single-channel input into 16 feature channels and is followed by batch normalization, a rectified linear unit (ReLU), and $2 \times 2$ max pooling. The second convolutional layer increases the number of channels to 32 and applies the same batch-normalization, ReLU, and max-pooling operations. The third convolutional layer produces 64 feature channels and is followed by ReLU activation and adaptive average pooling, which reduces the feature maps to $4 \times 4$.

The ReLU activation function is defined as

$$\mathsf{ReLU}(z) = \mathsf{max}(0, z). \tag{2}$$

The dimensions of the feature tensors throughout the network are

$$1 \times 32 \times 32 \to 16 \times 16 \times 16 \to 32 \times 8 \times 8 \to 64 \times 4 \times 4. \tag{3}$$

The pooled feature maps are flattened into a feature vector containing

$$64 \times 4 \times 4 = 1024 \tag{4}$$

elements. This vector is passed through a dropout layer with a probability of 0.25 to reduce excessive dependence on individual features. It is then processed by a fully connected layer containing 64 neurons and a ReLU activation. The final output layer contains seven neurons corresponding to the seven character classes. The classification stage can therefore be summarized as

$$1024 \to 64 \to 7. \tag{5}$$

Let $z_{i,c}$ denote the output score produced for class $c$ for the $i$th input sample. The corresponding class probability is calculated using the Softmax function:

$$p_{i,c} = \frac{\exp(z_{i,c})}{\sum_{j=1}^{K} \exp(z_{i,j})}, \tag{6}$$

where $K = 7$ is the total number of classes. The network is trained using the multiclass cross-entropy loss

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_{i,y_i}, \tag{7}$$

where $N$ is the number of samples in a training batch, $y_i$ is the ground-truth class of the $i$th sample, and $p_{i,y_i}$ is the probability assigned to that class. The Adam optimizer is used to update the network parameters according to the gradients of the cross-entropy loss.

The dataset split, number of training epochs, batch size, learning rate, random seed, and software and hardware environments are described in the Experimental Setup section. Classification accuracy, training time, and error analysis are presented in the Results and Discussion sections.

**Alternative CNN from the YJN branch** The integrated code also provides a second architecture selected with `--architecture yjn`. It uses three convolutional blocks with 16, 32, and 64 channels, each with batch normalization, ReLU, and max pooling. Global average pooling produces 64 features, followed by dropout with probability 0.3 and a direct classification layer. Both architectures use the same input resizing and stratified 75/25 training/test split through the shared training interface. The `baseline` option retains the architecture described above. Measured comparisons and configuration-selection results are reported below. Interpretation and recommendations remain subject to group review.

### 3.2.2 Subtask 7-2: Non-CNN Classifier

Subtask 7-2 uses a 32-by-32 unsigned-gradient HOG descriptor (4-pixel cells and nine orientation bins). An RBF SVM with $C$ selected on validation data is used when scikit-learn is present; the same interface falls back to a nearest-centroid classifier for a minimal installation. The feature extractor, split seed, classifier type, and accuracy are serialized so that the baseline can be compared with the CNN without changing the data protocol.

## 4 Experimental Setup

The recorded environment is `macOS-26.5.2-arm64-arm-64bit-Mach-O`. All experiments use CPU execution with one PyTorch thread. Seed 42 fixes the stratified 75/25 split: 5334 training and 1778 test samples. Twenty percent of the training portion is held out for validation during configuration selection. Four configurations per CNN compare direct resizing versus centered square padding, and learning rates 0.001 versus 0.0003. HOG-SVM compares the same preprocessing options and $C = 1$ versus $C = 10$. Each CNN is trained for 20 epochs with batch size 32. The highest validation accuracy selects the configuration (first trial breaks ties). The selected configuration is retrained on the full training portion.

Each of the seven classes contains 1016 samples. Sample paths, hashes and split indices are saved in `results/experiments/split_manifest.json`. Chip images and crops are never included in training. Timing is recorded with `perf_counter`; CNN preparation and SVM HOG extraction are excluded from training time. The original short smoke experiment previously inspected the same test partition, so this is a fixed protocol evaluation, not a new untouched

blind test. Only one seed is measured; uncertainty across seeds is not established. The random image split is not grouped by font or origin.

### 4.1 Configuration selection

| Model | Input | LR / C | Epochs | Validation accuracy (%) |
|---|---|---|---|---|
| baseline | resize | 0.001 | 20 | 99.72 |
| baseline | resize | 0.0003 | 20 | 99.44 |
| baseline | pad | 0.001 | 20 | 99.72 |
| baseline | pad | 0.0003 | 20 | 99.44 |
| yjn | resize | 0.001 | 20 | 96.99 |
| yjn | resize | 0.0003 | 20 | 96.90 |
| yjn | pad | 0.001 | 20 | 96.99 |
| yjn | pad | 0.0003 | 20 | 96.90 |
| hog-svm | resize | 1.0 | n/a | 97.74 |
| hog-svm | resize | 10.0 | n/a | 98.03 |
| hog-svm | pad | 1.0 | n/a | 97.74 |
| hog-svm | pad | 10.0 | n/a | 98.03 |

## 5 Results

### 5.1 Image-processing outputs

The integrated pipeline retains seven connected components after cleanup and splits wide regions into ten ordered boxes. Figure labels are manual reference annotations, not predicted labels. The group must verify crop quality and provide its own interpretation of the comparisons below.

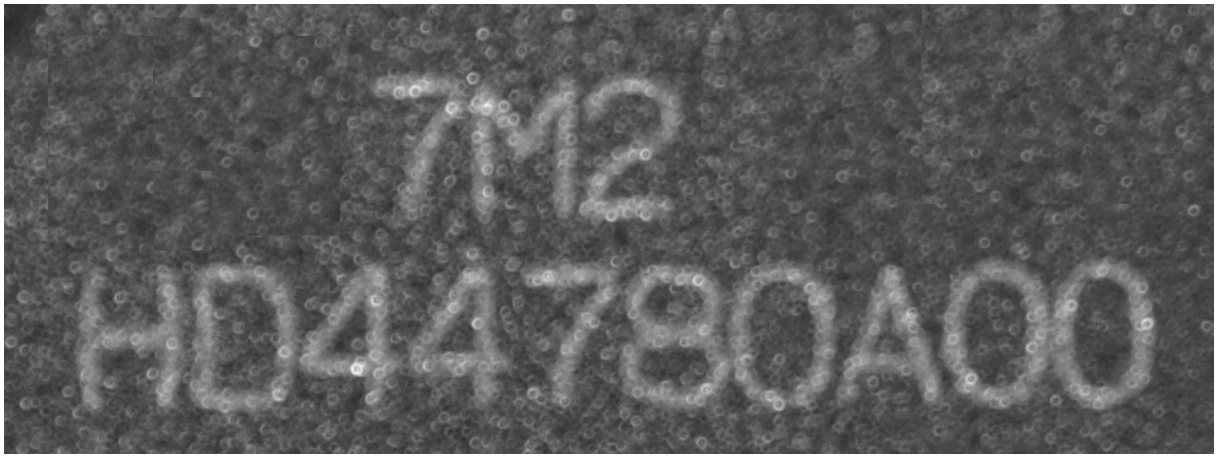



Figure 1: Original chip image.

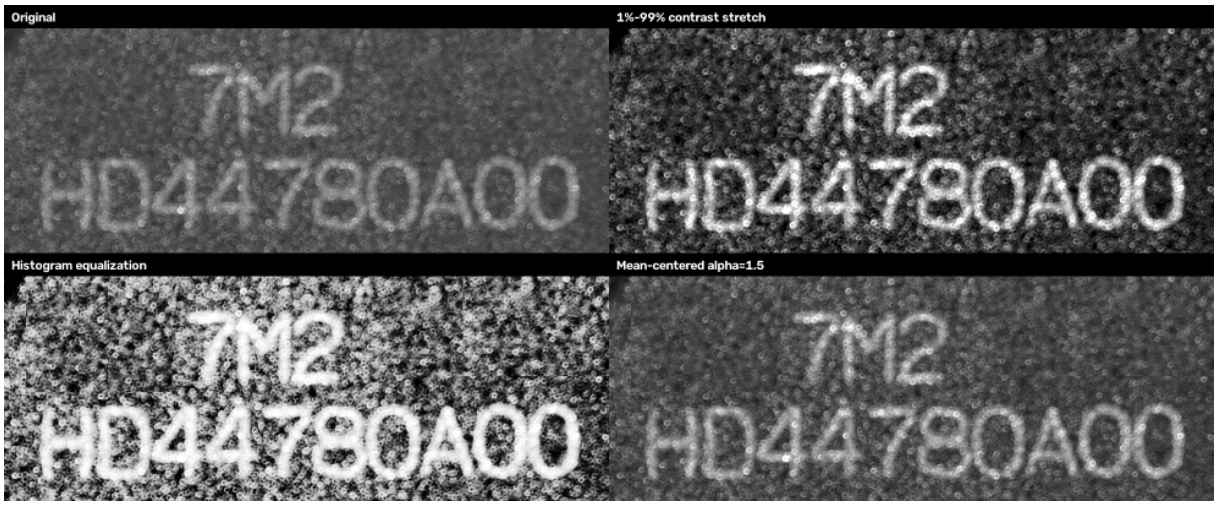



Figure 2: Task 1 enhancement comparison.

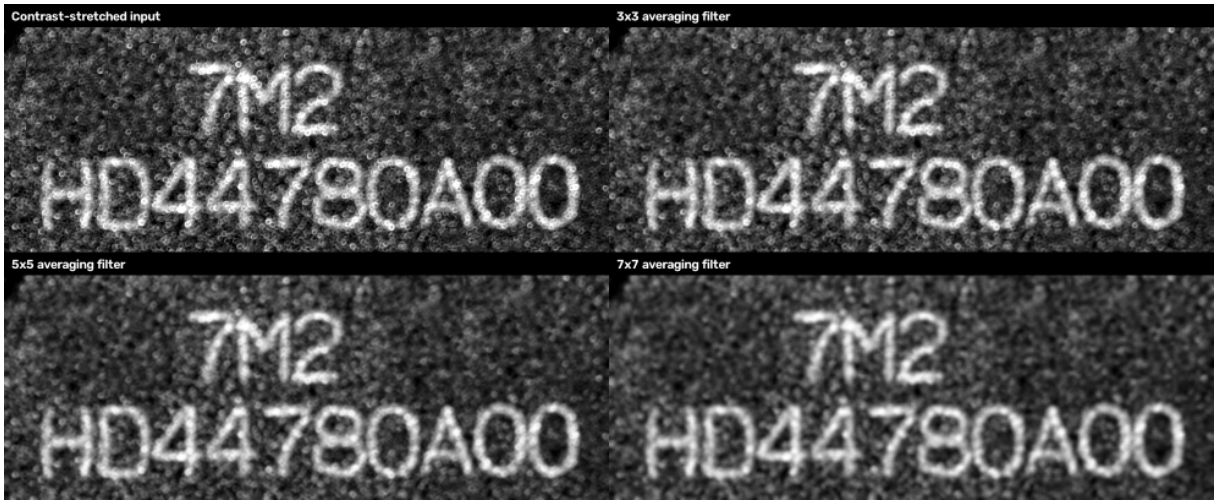



Figure 3: Task 2 averaging filter comparison.




Figure 4: Task 3 ROI comparison.




Figure 5: Task 4 thresholding comparison.

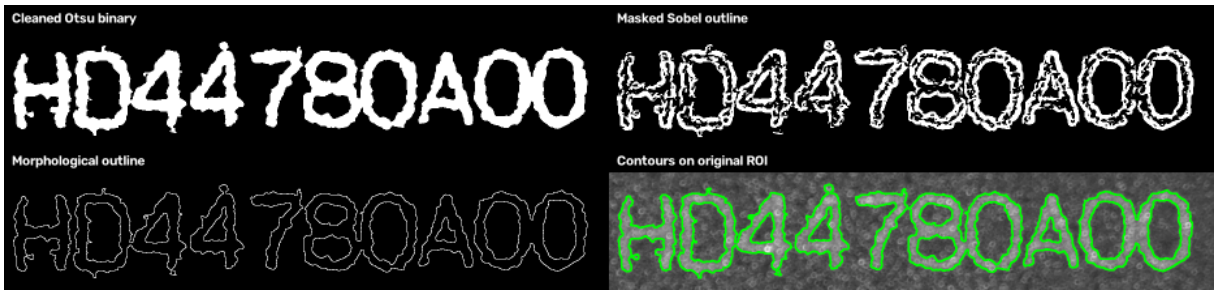



Figure 6: Task 5 outline comparison.




Figure 7: Task 6 crops with manual reference labels.

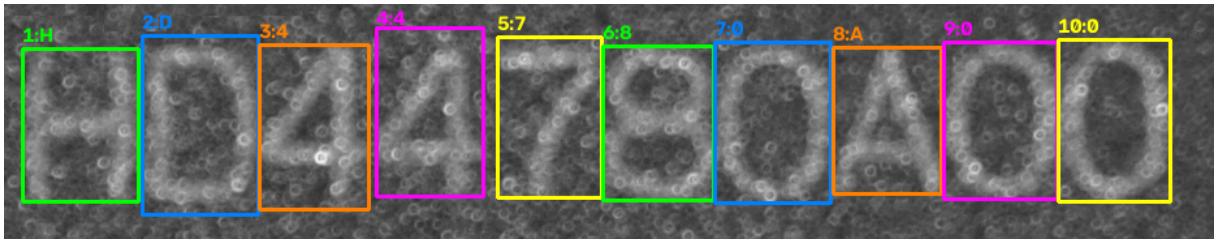



Figure 8: Task 6 ordered boxes with manual reference annotations.

## 5.2 Measured classification outputs

| Model | Test accuracy (%) | Training (s) | Chip prediction |
|---|---|---|---|
| baseline | 98.93 | 30.31 | HD4A780A00 |
| yjn | 92.69 | 33.69 | 8848888A88 |
| hog-svm | 98.37 | 0.63 | AAAAAAAAAA |

Chip inference inverts binary crops to dark glyphs on a white background, matching the training images. Baseline still misclassifies one target character; YJN and SVM fail on most chip crops despite high dataset accuracy. These are recorded failures, not successful end-to-end recognition. These values are automated measurements; interpretation and final conclusions must be supplied by the group. Training times exclude preprocessing and feature extraction and are not total pipeline costs.

## 5.3 Per-class evaluation

**baseline**

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| 0 | 1.000 | 0.988 | 0.994 | 254 |
| 4 | 1.000 | 0.984 | 0.992 | 254 |
| 7 | 0.992 | 1.000 | 0.996 | 254 |
| 8 | 1.000 | 0.988 | 0.994 | 254 |
| A | 0.977 | 0.992 | 0.984 | 254 |
| D | 0.981 | 0.992 | 0.986 | 254 |
| H | 0.976 | 0.980 | 0.978 | 254 |

Confusion matrix (rows: true, columns: predicted; order 0,4,7,8,A,D,H):

| | | | | | | |
|---|---|---|---|---|---|---|
| 251 | 0 | 0 | 0 | 0 | 2 | 1 |
| 0 | 250 | 0 | 0 | 2 | 0 | 2 |
| 0 | 0 | 254 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 251 | 1 | 2 | 0 |
| 0 | 0 | 0 | 0 | 252 | 0 | 2 |
| 0 | 0 | 0 | 0 | 1 | 252 | 1 |
| 0 | 0 | 2 | 0 | 2 | 1 | 249 |

**yjn**

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| 0 | 0.975 | 0.937 | 0.956 | 254 |
| 4 | 0.907 | 0.996 | 0.949 | 254 |
| 7 | 0.984 | 0.996 | 0.990 | 254 |
| 8 | 0.873 | 1.000 | 0.932 | 254 |
| A | 0.814 | 0.984 | 0.891 | 254 |
| D | 1.000 | 0.874 | 0.933 | 254 |
| H | 1.000 | 0.701 | 0.824 | 254 |

Confusion matrix (rows: true, columns: predicted; order 0,4,7,8,A,D,H):

| | | | | | | |
|---|---|---|---|---|---|---|
| 238 | 1 | 0 | 11 | 4 | 0 | 0 |
| 0 | 253 | 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 253 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 254 | 0 | 0 | 0 |
| 0 | 2 | 0 | 2 | 250 | 0 | 0 |
| 6 | 1 | 1 | 15 | 9 | 222 | 0 |
| 0 | 21 | 2 | 9 | 44 | 0 | 178 |

**hog-svm**

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| 0 | 0.992 | 0.988 | 0.990 | 254 |
| 4 | 1.000 | 0.996 | 0.998 | 254 |
| 7 | 0.996 | 0.988 | 0.992 | 254 |
| 8 | 0.992 | 0.976 | 0.984 | 254 |
| A | 0.937 | 0.996 | 0.966 | 254 |
| D | 0.988 | 0.980 | 0.984 | 254 |
| H | 0.984 | 0.961 | 0.972 | 254 |

Confusion matrix (rows: true, columns: predicted; order 0,4,7,8,A,D,H):

| | | | | | | |
|---|---|---|---|---|---|---|
| 251 | 0 | 0 | 2 | 1 | 0 | 0 |
| 0 | 253 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 251 | 0 | 1 | 0 | 2 |
| 1 | 0 | 0 | 248 | 4 | 1 | 0 |
| 1 | 0 | 0 | 0 | 253 | 0 | 0 |
| 0 | 0 | 0 | 0 | 4 | 249 | 1 |
| 0 | 0 | 1 | 0 | 7 | 2 | 244 |

### 5.4 Dataset and split accounting

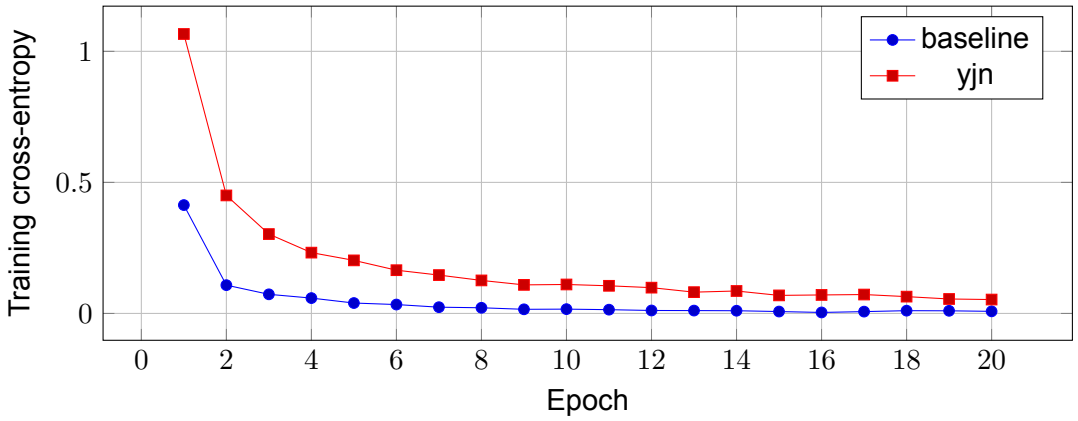
The classification dataset contains 7,112 images. Each class has 1,016 images, with 762 assigned to training and 254 to testing. Within the training portion, each class contributes 610 fitting samples and 152 validation samples: 4,270 fitting and 1,064 validation samples in total. The final models are refitted on all 5,334 training images. No augmentation is applied. Images are converted to grayscale and scaled to $[0, 1]$ for CNN input. The padding alternative centers the image on a square canvas before nearest-neighbor resizing to $32 \times 32$. The original training images are already square, so this padding operation does not change their geometry; equal validation scores do not establish that padding has no effect for differently shaped inputs.

### 5.5 Recorded software versions

platform: macOS-26.5.2-arm64-arm-64bit-Mach-O
python: 3.13.13
torch: 2.14.1
numpy: 2.5.4

### 5.6 Final CNN training loss

The curves show training loss only. They do not measure held-out generalization and are not used to select an epoch using the test data.




### 5.7 Chip prediction audit

The table below compares actual predictions with the known reference text. Reference labels come from the assignment and were not inferred from filenames. Binary crops are inverted before classification to match the dark-glyph, white-background polarity of the training images. The uncorrected predictions are retained in the JSON for traceability. No training or parameter selection uses these ten crops.

| Position | Reference | Baseline | YJN | HOG-SVM |
|---|---|---|---|---|
| 1 | H | H | 8 | A |
| 2 | D | D | 8 | A |
| 3 | 4 | 4 | 4 | A |
| 4 | 4 | A | 8 | A |
| 5 | 7 | 7 | 8 | A |
| 6 | 8 | 8 | 8 | A |
| 7 | 0 | 0 | 8 | A |
| 8 | A | A | A | A |
| 9 | 0 | 0 | 8 | A |
| 10 | 0 | 0 | 8 | A |

baseline: 9/10 position matches; exact-line match: false. yjn: 3/10 position matches; exact-line match: false. hog-svm: 1/10 position matches; exact-line match: false.

### 5.8 Evidence boundaries requiring group review

The dataset test scores and chip predictions are separate evaluation surfaces. The tables record high dataset accuracy together with errors on the chip line. They do not determine the cause of those errors. The group must inspect segmentation quality, appearance differences and preprocessing choices before attributing a failure to a particular stage. Only one image and one split seed are evaluated. The recorded image-level split does not establish independence across fonts or source images. The earlier smoke run inspected the same test partition, and the present evaluation must not be described as a new blind test.

### 5.9 Timing scope

CNN training measurements include optimization but exclude loading and preparing the input tensor. SVM fitting measurements exclude HOG extraction. Therefore the listed training times are fitting costs, not end-to-end processing costs. Initial chip timing fields were removed after polarity correction because they referred to the uncorrected inference run. No corrected latency benchmark is claimed.

## 6 Discussion

Group-authored analysis pending: explain the measured CNN/non-CNN differences, preprocessing and hyperparameter sensitivity, robustness and failure cases. The automated tables are evidence for review, not an authored explanation.

## 7 Conclusion

Group-authored conclusion pending: state the findings supported by the measurements, lessons learned and limitations. Identify improvements justified by the group.

## Code Execution Guide

Install Python 3.13 and the dependencies in `requirements.txt`. Place the official classification folders and chip image under `dataset_2026/`. From the repository root run:

```
python segment.py
python scripts/run_experiments.py --epochs 20
python scripts/predict_characters.py \
  --model results/models/baseline_final.pt
python scripts/predict_characters.py \
  --model results/models/hog_svm_final.pkl
python scripts/build_report_results.py
python scripts/enrich_report.py
python -m pytest -q
make pdf
```

The chip crops in `results/figures/characters/` are inference inputs only. They must not be inserted into the classifier training dataset. See the repository README for environment setup, input provenance, validation protocol and timing boundaries. Model checkpoints are generated locally by the experiment command.

## Acknowledgment and Provenance

The code uses Python, NumPy, Pillow, OpenCV, PyTorch and scikit-learn. The integrated image-processing implementation is from PR 1 by Jiayi Guo; the alternative CNN is from the YJN branch; the expanded report draft was contributed by Hongchao Wang. The group must verify individual contributions, external code citations and actual AI assistance before submission. The course permits LLM language editing, while ideas, analysis and reasoning must originate from group members. Automated numerical outputs in this working report are not a substitute for group-authored interpretation.

## References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.